Class 10: Structural Bioinformatics 1

Harshitha (PID: A17775151)

Table of contents

Background

The main repository of high-resolution structural data on biomolecules is called the **Protein Data Bank** (PDB).

PDB statistics

What is in the PDB in terms of molecule type and structure determination method?

Read a CSV file of current PDB stats obtained from https://www.rcsb.org/stats/summary

```
pdb <- read.csv("Data Export Summary.csv")
pdb
```

```
           Molecular.Type   X.ray     EM    NMR Integrative Multiple.methods
1          Protein (only) 180,758 23,111 12,813         348               229
2 Protein/Oligosaccharide  10,488  3,741     34           8                11
3              Protein/NA   9,205  6,751    287          26                 8
4     Nucleic acid (only)   3,154    250  1,578           3                15
5                   Other     178     27     35           4                 0
6  Oligosaccharide (only)      11      0      6           0                 1
  Neutron Other   Total
1      84    32 217,375
2       1     0  14,283
3       0     0  16,277
4       3     1   5,004
5       0     0     244
6       0     4      22
```

> Q1: What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy.

80.49% of structures in the PDB are solved by X-ray, and 13.38% are solved by Electron Microscopy.

```
pdb$X.ray
```

```
[1] "180,758" "10,488"  "9,205"   "3,154"   "178"     "11"
```

This print out above `pdb$X.ray` is "character" not "numeric". Therefore I can't do math with it.

Two functions that can help here are `sub()` and `as.numeric()`

```
# We want to get rid (or sub out) commas:
x <- pdb$X.ray
tmp <- sub(",", "", x)
sum( as.numeric(tmp) )
```

```
[1] 203794
```

We could make a function to do this:

```
rm.comma <- function(x) {
  tmp <- sub(",", "", x)
  sum( as.numeric(tmp) )
}
```

```
# Percentage of structures solved by X-ray
rm.comma(pdb$X.ray) / rm.comma(pdb$Total)
```

```
[1] 0.8048577
```

```
# Percentage of structures solved by EM
rm.comma(pdb$EM) / rm.comma(pdb$Total)
```

```
[1] 0.1338046
```

We could also use a different import function for this CSV that speaks American (i.e. deals with commas in numbers in a comma separated value file).

```
library(readr)

pdb <- read_csv("Data Export Summary.csv")
```

```
Rows: 6 Columns: 9
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (1): Molecular Type
dbl (4): Integrative, Multiple methods, Neutron, Other
num (4): X-ray, EM, NMR, Total

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
pdb
```

```
# A tibble: 6 x 9
  `Molecular Type`    `X-ray`    EM   NMR Integrative `Multiple methods` Neutron
  <chr>                 <dbl> <dbl> <dbl>       <dbl>                <dbl>   <dbl>
1 Protein (only)       180758 23111 12813         348                  229      84
2 Protein/Oligosacch~   10488  3741    34           8                   11       1
```

```
3 Protein/NA              9205  6751   287          26                  8       0
4 Nucleic acid (only)     3154   250  1578           3                 15       3
5 Other                    178    27    35           4                  0       0
6 Oligosaccharide (o~       11     0     6           0                  1       0
# i 2 more variables: Other <dbl>, Total <dbl>
```

```
n.tot <- sum(pdb$Total)
n.xray <- sum(pdb$`X-ray`)
n.em <- sum(pdb$EM)

# Percentage of structures solved by X-ray
(n.xray / n.tot) * 100
```

```
[1] 80.48577
```

```
# Percentage of structures solved by EM
(n.em / n.tot) * 100
```

```
[1] 13.38046
```

Q. How many total protein structures are there in the dataset?

There are 217375 protein structures in the database.

```
pdb$Total[1]
```

```
[1] 217375
```

Q2: What proportion of structures in the PDB are protein?

0.107% of PDB structures are protein.

```
# The total number of entries in UniProt is 202,556,314.
(pdb$Total[1] / 202556314) * 100
```

```
[1] 0.1073158
```

Key-point: We have a very, very small structural coverage of known proteins (~0.1%). Most structures we know about (~80%) come from one method (X-ray crystallography).

SKIPPED IN CLASS Q3: Type HIV in the PDB website search box on the home page and determine how many HIV-1 protease structures are in the current PDB?

Visualizing PDB data with Mol-star

Main stand alone web version with all features is at https://molstar.org/viewer/

Figure 1: The HIV-protease enzyme is a homodimer with two chains

Getting to know HIV-Pr

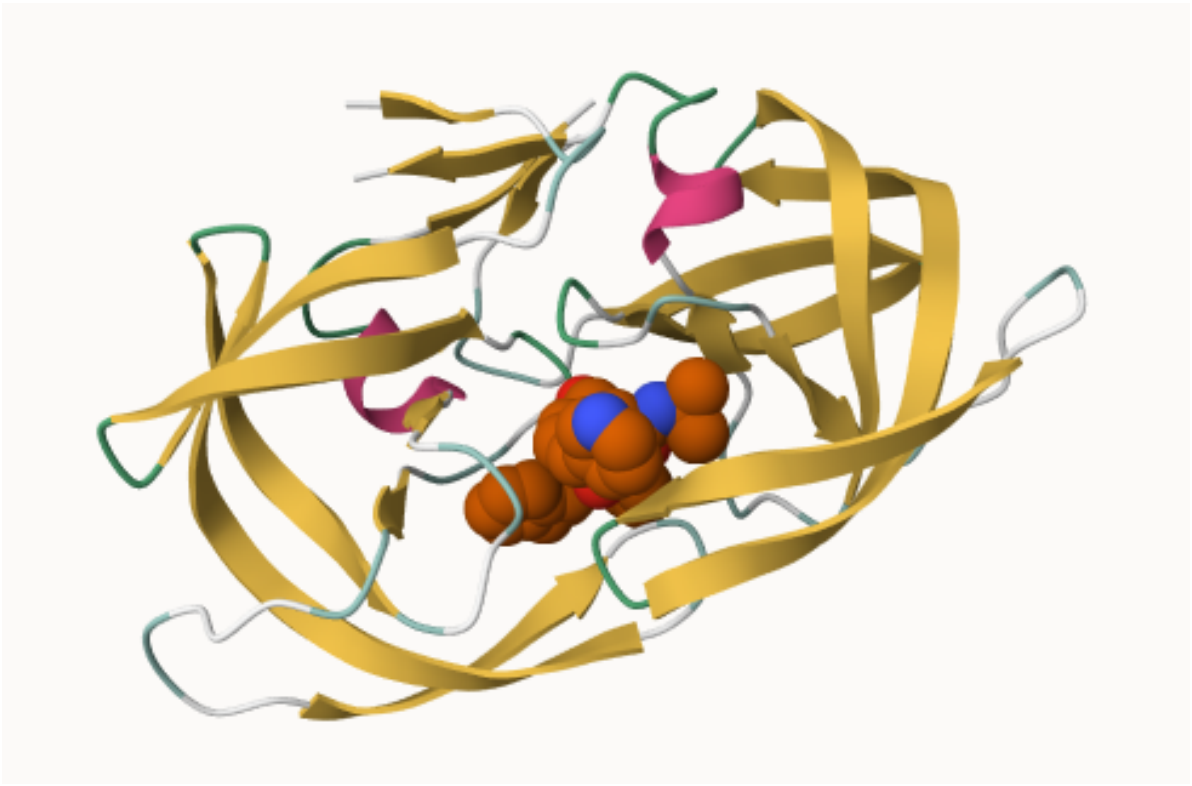


Figure 2: Ligand with Spacefill representation, and polymer colored by secondary structure

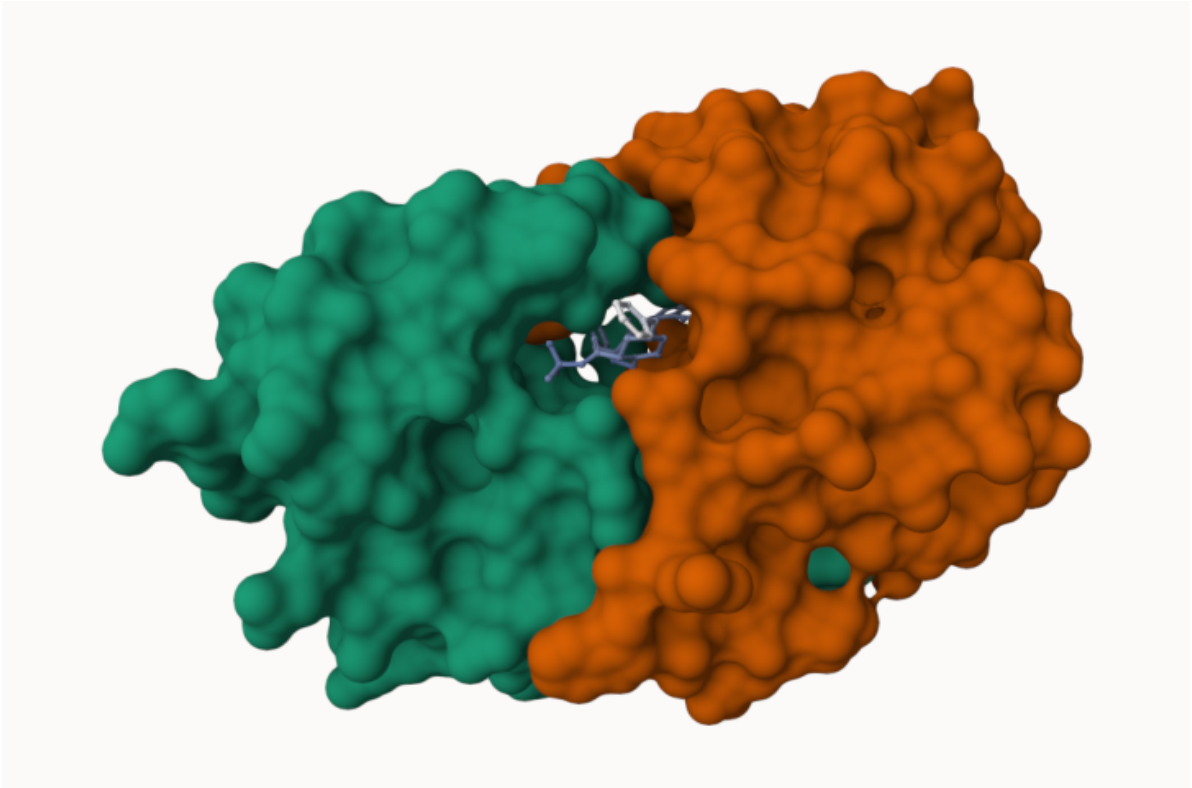


Figure 3: Surface display showing the binding cleft site for the inhibitor (drug)

Delving deeper

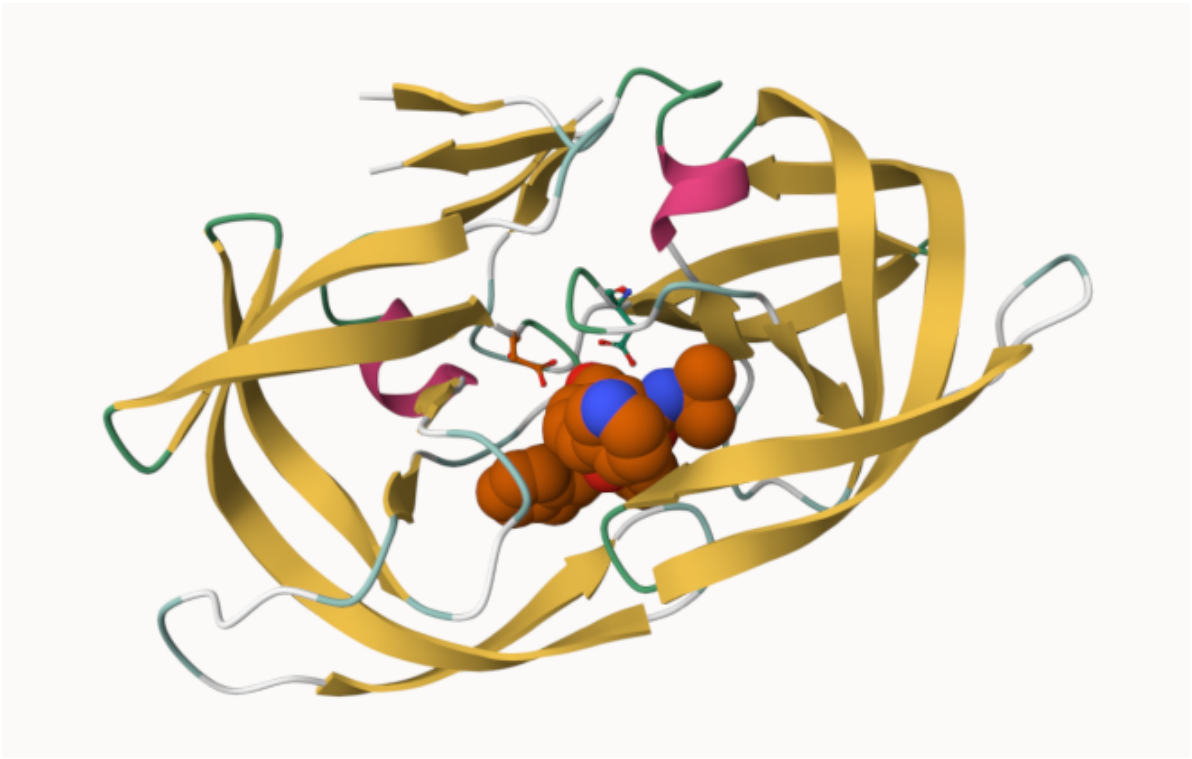


Figure 4: Display showing the Asp 25 (D25) residue in each chains, which is critical to protease activity. The polymer is colored by secondary structure.

The important role of water

> Q4: Water molecules normally have 3 atoms. Why do we see just one atom per water molecule in this structure?

We only see each water molecule's oxygen atom. X-ray crystallography has a 2.00 Angstrom resolution, and hydrogen is smaller than this – thus, it does not appear in the structure.

> Q5: There is a critical "conserved" water molecule in the binding site. Can you identify this water molecule? What residue number does this water molecule have?

The water molecule has the residue number 308.

> Q6: Generate and save a figure clearly showing the two distinct chains of HIV-protease along with the ligand. You might also consider showing the catalytic

residues ASP 25 in each chain and the critical water. Add this figure to your Quarto document.

Discussion Topic: Can you think of a way in which indinavir, or even larger ligands and substrates, could enter the binding site?

Figure 5: Spacefill display of catalytic ASP25 amino acids and key binding site water molecule. Ligand is displayed with a ball-and-stick representation.

For indinavir, or other ligands/substrates to enter the binding site, it's possible that the two loops at the top of the protease structure (as shown in the image above; one loop per chain) undergo a conformational change and "open" in a gate-like manner to enable substrate entry. Once the substrate enters, the loops return to their original, "closed" conformation. The positioning of the loops and the newly-entered substrate may be stabilized by non-covalent/hydrogen bonding interactions with the critical water molecule (HOH 308, shown in the image above). Furthermore, the substrate is stabilized by interactions with the catalytic Asp 25 residues in the protease's active site.

Getting started with the Bio3D package

The following code has been inputted into the R brain, to install useful packages. Pak can install from multiple sources (i.e. "bioboot/bio3dview" from GitHub, "NGLVieweR" from CRAN, "bioc::msa" from Bioconductor).

```
# install.packages("pak")
# pak::pkg_install( c("bioboot/bio3dview", "NGLVieweR", "bioc::msa"))
```

Bio3D is an R package from CRAN for structural bioinformatics.

```
library(bio3d)

pdb <- read.pdb("1hsg")
```

```
  Note: Accessing on-line PDB file
```

```
pdb
```

```
 Call:  read.pdb(file = "1hsg")

   Total Models#: 1
     Total Atoms#: 1686,  XYZs#: 5058  Chains#: 2  (values: A B)

     Protein Atoms#: 1514  (residues/Calpha atoms#: 198)
     Nucleic acid Atoms#: 0  (residues/phosphate atoms#: 0)

     Non-protein/nucleic Atoms#: 172  (residues: 128)
     Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]

   Protein sequence:
      PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
      QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
      ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
      VNIIGRNLLTQIGCTLNF

+ attr: atom, xyz, seqres, helix, sheet,
         calpha, remark, call
```

Q7: How many amino acid residues are there in this pdb object?

There are 198 amino acid residues.

Q8: Name one of the two non-protein residues?

One non-protein residue is HOH (res no. 127). The other is MK1 (res no. 1).

Q9. How many protein chains are in this structure?

There are two protein chains.

```
# To access dataset's attributes
attributes(pdb)
```

```
$names
[1] "atom"   "xyz"    "seqres" "helix"  "sheet"  "calpha" "remark" "call"

$class
[1] "pdb" "sse"
```

```
# Access the atom attribute in pdb
head(pdb$atom)
```

```
  type eleno elety  alt resid chain resno insert      x      y     z o     b
1 ATOM     1     N <NA>   PRO     A     1   <NA> 29.361 39.686 5.862 1 38.10
2 ATOM     2    CA <NA>   PRO     A     1   <NA> 30.307 38.663 5.319 1 40.62
3 ATOM     3     C <NA>   PRO     A     1   <NA> 29.760 38.071 4.022 1 42.64
4 ATOM     4     O <NA>   PRO     A     1   <NA> 28.600 38.302 3.676 1 43.40
5 ATOM     5    CB <NA>   PRO     A     1   <NA> 30.508 37.541 6.342 1 37.87
6 ATOM     6    CG <NA>   PRO     A     1   <NA> 29.296 37.591 7.162 1 38.40
  segid elesy charge
1  <NA>     N   <NA>
2  <NA>     C   <NA>
3  <NA>     C   <NA>
4  <NA>     O   <NA>
5  <NA>     C   <NA>
6  <NA>     C   <NA>
```

There are lots of functions that can work with these `pdb` objects:

```
head(pdbseq(pdb))
```

```
  1   2   3   4   5   6
"P" "Q" "I" "T" "L" "W"
```

Quick PDB visualization in R

We can have a quick view of any of these `pdb` objects:

```
library(bio3dview)
library(NGLVieweR)

view.pdb(pdb) |>
  setSpin()
```

Let's try a custom view:

```
# To color by secondary structure
view.pdb(pdb,
         colorScheme = "sse",
         backgroundColor = "black")
```

> Q. Create a custom view of HIV-Pr highlighting the active site ASP residues (`resno=25`), the two chains (in your choice of colors), and the ligand all on a custom color background.

```
# Select the ASP 25 residue
active.site <- atom.select(pdb, resno = 25)

# Highlight D25 residues in spacefill representation
view.pdb(pdb,
         cols = c("green", "teal"),
         highlight = active.site,
         highlight.style = "spacefill",
         backgroundColor = "black") |>
  setRock()
```

Predict the flexibility of a given structure

Let's do a Normal Mode Analysis (NMA) to predict the flexibility of a given `pdb` object:

```
# Step 1: Read in PDB structure of Adenylate Kinase
adk <- read.pdb("6s36")
```

```
  Note: Accessing on-line PDB file
   PDB has ALT records, taking A only, rm.alt=TRUE
```

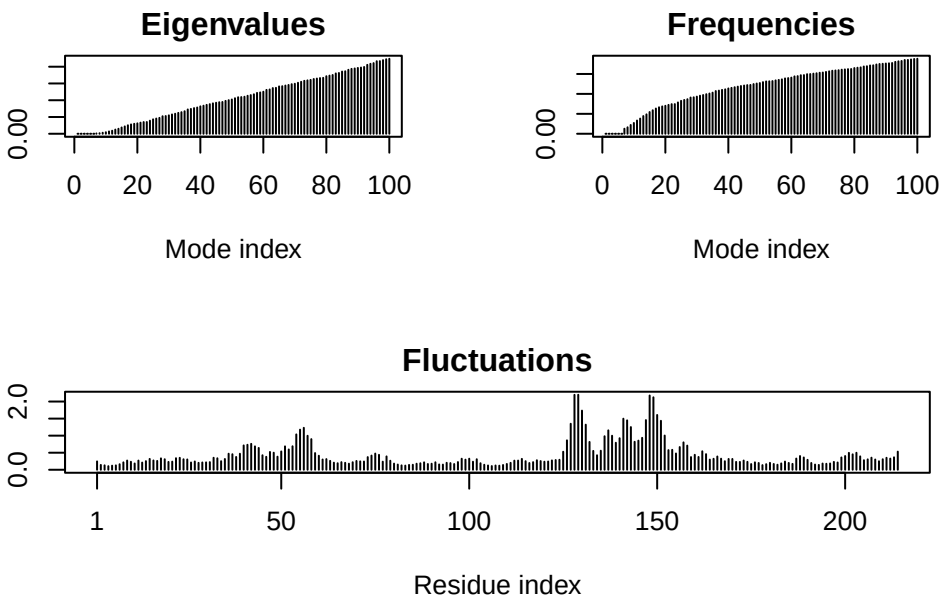
```
adk
```

```
 Call:  read.pdb(file = "6s36")

   Total Models#: 1
     Total Atoms#: 1898,  XYZs#: 5694  Chains#: 1  (values: A)

     Protein Atoms#: 1654  (residues/Calpha atoms#: 214)
     Nucleic acid Atoms#: 0  (residues/phosphate atoms#: 0)

     Non-protein/nucleic Atoms#: 244  (residues: 244)
     Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1) ]

   Protein sequence:
      MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLVT
      DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDKI
      VGRRVHAPSGRVYHVKFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
      YYSKEAEAGNTKYAKVDGTKPVAEVRADLEKILG

+ attr: atom, xyz, seqres, helix, sheet,
        calpha, remark, call
```

```
# Step 2: Pass pdb object to nma() function
m <- nma(adk)
```

```
 Building Hessian...        Done in 0.017 seconds.
 Diagonalizing Hessian...   Done in 0.413 seconds.
```

```
# Step 3a: Pass nma results to plot(). Fluctuations plot is particularly interesting!
plot(m)
```



View the results with an interactive structure view (use `view.nma()`):

```
# Step 3b: Pass nma results to view.nma()
view.nma(m)
```

Write out the results for viewing in Mol-star:

```
# Step 3c: Pass nma results to mktrj(), to generate file to open in Mol-star
mktrj(m, file="nma.pdb")
```

Figure 6: Mol-star view of Adenylate Kinase. Playing the movie in Mol* shows a highly dynamic, "opening/closing" motion of the structure.

Comparative analysis of the ADK family

Setup

The following installations are relevant for this lab, and were already done in the R console:

```
# install.packages("bio3d")
# install.packages("NGLVieweR")

# install.packages("remotes")
# remotes::install_github("bioboot/bio3dview")

# install.packages("BiocManager")
# BiocManager::install("msa")
```

Q10. Which of the packages above is found only on BioConductor and not CRAN?

"msa" is found only on BioConductor (must be installed using `BiocManager::install()`).

Q11. Which of the above packages is not found on BioConductor or CRAN?:

"bio3dview" is not found on BioConductor or CRAN (found on GitHub instead, and must be installed using `devtools::install_github`).

Q12. True or False? Functions from the pak package can be used to install packages from GitHub and BitBucket?

True! Pak can install from multiple sources.

Search and retrieve ADK structures

Our first step is find a sequence for this family. We will use the database ID "1ake_A" here:

```
id <- "1ake_A"

# Step 1: Use get.seq() with input database identifier.
aa <- get.seq(id)
```

```
Warning in get.seq(id): Removing existing file: seqs.fasta
```

```
Fetching... Please wait. Done.
```

```
aa
```

```
              1        .         .         .         .         .         60
pdb|1AKE|A    MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLVT
              1        .         .         .         .         .         60

             61        .         .         .         .         .         120
pdb|1AKE|A    DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDRI
             61        .         .         .         .         .         120

            121        .         .         .         .         .         180
pdb|1AKE|A    VGRRVHAPSGRVYHVKFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
            121        .         .         .         .         .         180

            181        .         .         .   214
pdb|1AKE|A    YYSKEAEAGNTKYAKVDGTKPVAEVRADLEKILG
            181        .         .         .   214

Call:
  read.fasta(file = outfile)

Class:
  fasta

Alignment dimensions:
  1 sequence rows; 214 position columns (214 non-gap, 0 gap)

+ attr: id, ali, call
```

Q13. How many amino acids are in this sequence, i.e. how long is this sequence?

There are 214 amino acids in the sequence.

Search for related sequences in the database

```
# Step 2a: Find related sequences via a PDB search, using blast.pdb() function
blast <- blast.pdb(aa)
```

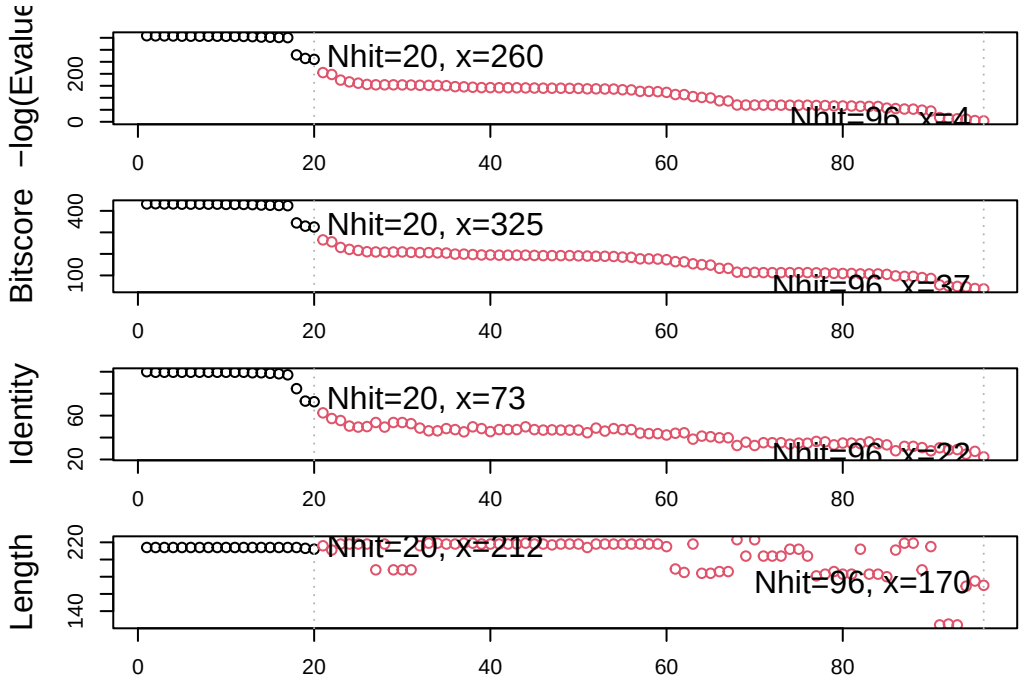
```
 Searching ... please wait (updates every 5 seconds) RID = 020ZJ0B9014
 ...
 Reporting 96 hits
```

```
# Preview the top hits
head(blast$hit.tbl)
```

```
       queryid subjectids identity alignmentlength mismatches gapopens q.start
1 Query_229329     1AKE_A  100.000             214          0        0       1
2 Query_229329     8BQF_A   99.533             214          1        0       1
3 Query_229329     4X8M_A   99.533             214          1        0       1
4 Query_229329     6S36_A   99.533             214          1        0       1
5 Query_229329     9R6U_A   99.533             214          1        0       1
6 Query_229329     9R71_A   99.533             214          1        0       1
  q.end s.start s.end    evalue bitscore positives mlog.evalue pdb.id    acc
1   214       1   214 1.82e-156      432    100.00    358.6044 1AKE_A 1AKE_A
2   214      21   234 2.97e-156      433    100.00    358.1147 8BQF_A 8BQF_A
3   214       1   214 3.25e-156      432    100.00    358.0246 4X8M_A 4X8M_A
4   214       1   214 4.77e-156      432    100.00    357.6409 6S36_A 6S36_A
5   214       1   214 1.06e-155      431     99.53    356.8424 9R6U_A 9R6U_A
6   214       1   214 1.25e-155      431     99.53    356.6775 9R71_A 9R71_A
```

```
# Plot BLAST results
hits <- plot(blast)
```

```
  * Possible cutoff values:    260 3
            Yielding Nhits:    20 96

  * Chosen cutoff value of:    260
            Yielding Nhits:    20
```




```
# Each dot is one row from hits table.
# Dashed line represents point of largest drop-off in norm. scores (E-value jump)
# Black dots to the left of dashed line are considered good hits
# Red dots to the right of dashed line have high E-values
```

```
# List out the top hits
hits$pdb.id
```

```
 [1] "1AKE_A" "8BQF_A" "4X8M_A" "6S36_A" "9R6U_A" "9R71_A" "8Q2B_A" "8RJ9_A"
 [9] "6RZE_A" "4X8H_A" "3HPR_A" "1E4V_A" "5EJE_A" "1E4Y_A" "3X2S_A" "6HAP_A"
[17] "6HAM_A" "8PVW_A" "4K46_A" "4NP6_A"
```

```
#Step 2b: Download BLAST hits with get.pdb() function.
files <- get.pdb(hits$pdb.id, path="pdbs", split=TRUE, gzip=TRUE)
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1AKE.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8BQF.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4X8M.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6S36.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/9R6U.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/9R71.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8Q2B.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8RJ9.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6RZE.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4X8H.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3HPR.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4V.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/5EJE.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4Y.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3X2S.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAP.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAM.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8PVW.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4K46.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4NP6.pdb.gz exists. Skipping download


  |
  |                                                                      |   0%
  |
  |====                                                                  |   5%
  |
  |=======                                                               |  10%
  |
  |==========                                                            |  15%
  |
  |==============                                                        |  20%
  |
  |==================                                                    |  25%
  |
  |======================                                                |  30%
  |
  |=========================                                             |  35%
  |
  |============================                                          |  40%
  |
  |================================                                      |  45%
  |
  |===================================                                   |  50%
  |
  |======================================                                |  55%
  |
```

```
  |=======================================                               |  60%
  |
  |============================================                          |  65%
  |
  |================================================                      |  70%
  |
  |====================================================                  |  75%
  |
  |========================================================              |  80%
  |
  |============================================================          |  85%
  |
  |================================================================      |  90%
  |
  |====================================================================  |  95%
  |
  |======================================================================| 100%
```

Align and superpose structures

Align and superpose all these ADK structures

```
# Step 3: Align related PDBs using pdbaln() function, in superposed manner
pdbs <- pdbaln(files, fit = TRUE, exefile="msa")
```

```
Reading PDB files:
pdbs/split_chain/1AKE_A.pdb
pdbs/split_chain/8BQF_A.pdb
pdbs/split_chain/4X8M_A.pdb
pdbs/split_chain/6S36_A.pdb
pdbs/split_chain/9R6U_A.pdb
pdbs/split_chain/9R71_A.pdb
pdbs/split_chain/8Q2B_A.pdb
pdbs/split_chain/8RJ9_A.pdb
pdbs/split_chain/6RZE_A.pdb
pdbs/split_chain/4X8H_A.pdb
pdbs/split_chain/3HPR_A.pdb
pdbs/split_chain/1E4V_A.pdb
pdbs/split_chain/5EJE_A.pdb
pdbs/split_chain/1E4Y_A.pdb
pdbs/split_chain/3X2S_A.pdb
```

```
pdbs/split_chain/6HAP_A.pdb
pdbs/split_chain/6HAM_A.pdb
pdbs/split_chain/8PVW_A.pdb
pdbs/split_chain/4K46_A.pdb
pdbs/split_chain/4NP6_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
....   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..

Extracting sequences

pdb/seq: 1   name: pdbs/split_chain/1AKE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 2   name: pdbs/split_chain/8BQF_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 3   name: pdbs/split_chain/4X8M_A.pdb
pdb/seq: 4   name: pdbs/split_chain/6S36_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 5   name: pdbs/split_chain/9R6U_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 6   name: pdbs/split_chain/9R71_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 7   name: pdbs/split_chain/8Q2B_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 8   name: pdbs/split_chain/8RJ9_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 9   name: pdbs/split_chain/6RZE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 10   name: pdbs/split_chain/4X8H_A.pdb
pdb/seq: 11   name: pdbs/split_chain/3HPR_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 12   name: pdbs/split_chain/1E4V_A.pdb
```

```
pdb/seq: 13   name: pdbs/split_chain/5EJE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 14   name: pdbs/split_chain/1E4Y_A.pdb
pdb/seq: 15   name: pdbs/split_chain/3X2S_A.pdb
pdb/seq: 16   name: pdbs/split_chain/6HAP_A.pdb
pdb/seq: 17   name: pdbs/split_chain/6HAM_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 18   name: pdbs/split_chain/8PVW_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 19   name: pdbs/split_chain/4K46_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 20   name: pdbs/split_chain/4NP6_A.pdb
```

```
# pdbs
```

Optional: Viewing our superposed structures

Quick interactive structural view

```
view.pdbs(pdbs)
```

Annotate collected PDB structures

We can use `pdb.annotate()` to annotate PDB files.

```
# Vector with PDB database codes of collected structures
ids <- basename.pdb(pdbs$id)

# Here, we use pdb.annotate() to annotate structure to source species.
anno <- pdb.annotate(ids)
unique(anno$source)
```

```
[1] "Escherichia coli"
[2] "Escherichia coli K-12"
[3] "Escherichia coli O139:H28 str. E24377A"
[4] "Escherichia coli str. K-12 substr. MDS42"
[5] "Photobacterium profundum"
[6] "Vibrio cholerae O1 biovar El Tor str. N16961"
```

```
# Columns of anno are the available annotation features.
head(anno)
```

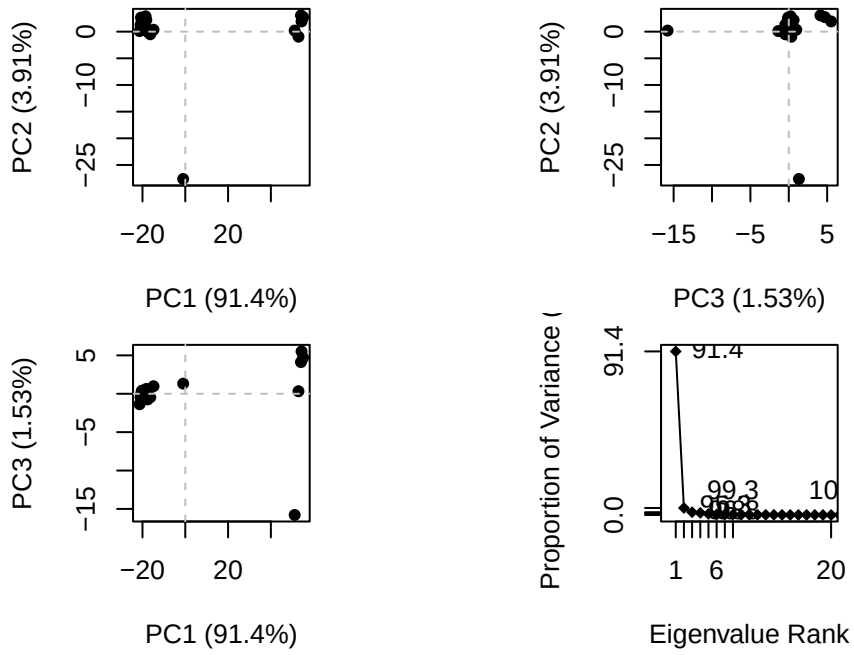
```
       structureId chainId macromoleculeType chainLength experimentalTechnique
1AKE_A        1AKE       A           Protein         214                  X-
ray
8BQF_A        8BQF       A           Protein         234                  X-
ray
4X8M_A        4X8M       A           Protein         214                  X-
ray
6S36_A        6S36       A           Protein         214                  X-
ray
9R6U_A        9R6U       A           Protein         214                  X-
ray
9R71_A        9R71       A           Protein         214                  X-
ray
       resolution       scopDomain                   pfam         ligandId
1AKE_A       2.00 Adenylate kinase Adenylate kinase (ADK)              AP5
8BQF_A       2.05             <NA> Adenylate kinase (ADK)              AP5
4X8M_A       2.60             <NA> Adenylate kinase (ADK)             <NA>
6S36_A       1.60             <NA> Adenylate kinase (ADK) CL (3),NA,MG (2)
9R6U_A       1.77             <NA> Adenylate kinase (ADK)       AP5,GOL,NA
9R71_A       1.61             <NA> Adenylate kinase (ADK)              AP5
                                                ligandName           source
1AKE_A                     BIS(ADENOSINE)-5'-PENTAPHOSPHATE Escherichia coli
8BQF_A                     BIS(ADENOSINE)-5'-PENTAPHOSPHATE Escherichia coli
4X8M_A                                                 <NA> Escherichia coli
6S36_A        CHLORIDE ION (3),SODIUM ION,MAGNESIUM ION (2) Escherichia coli
9R6U_A BIS(ADENOSINE)-5'-PENTAPHOSPHATE,GLYCEROL,SODIUM ION Escherichia coli
9R71_A                     BIS(ADENOSINE)-5'-PENTAPHOSPHATE Escherichia coli

1AKE_A STRUCTURE OF THE COMPLEX BETWEEN ADENYLATE KINASE FROM ESCHERICHIA COLI AND THE INHIBI
8BQF_A
4X8M_A
6S36_A
9R6U_A                                                                        C
9R71_A                                                                      Crys
                                                citation rObserved   rFree
1AKE_A            Muller, C.W., et al. J Mol Biology (1992)   0.19600      NA
8BQF_A Scheerer, D., et al. Proc Natl Acad Sci U S A (2023)   0.22073 0.25789
4X8M_A              Kovermann, M., et al. Nat Commun (2015)   0.24910 0.30890
6S36_A                Rogne, P., et al. Biochemistry (2019)   0.16320 0.23560
```

```
9R6U_A             Mattsson, J., et al. Biochemistry (2025)        NA 0.22790
9R71_A             Mattsson, J., et al. Biochemistry (2025)   0.19600 0.24400
         rWork spaceGroup
1AKE_A 0.19600  P 21 2 21
8BQF_A 0.21882  P 2 21 21
4X8M_A 0.24630    C 1 2 1
6S36_A 0.15940    C 1 2 1
9R6U_A 0.19190  P 21 2 21
9R71_A 0.19300  P 21 21 2
```

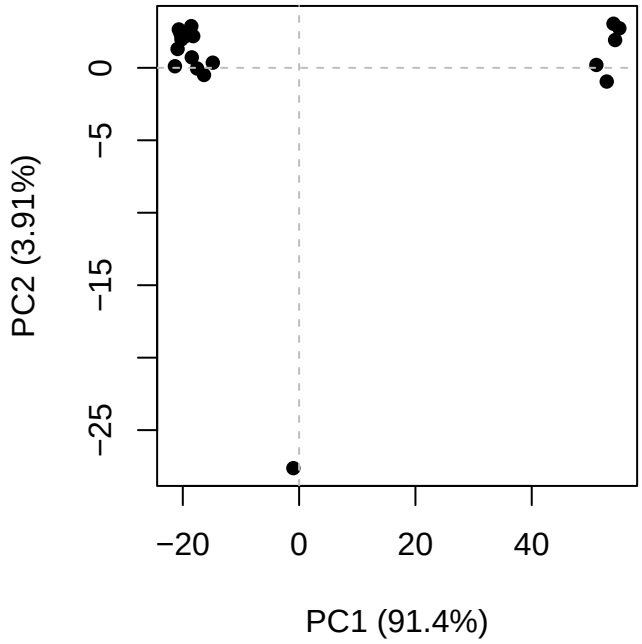
Principal component analysis

PCA of all this structural data (x, y, and z atom coordinates):

```
# Step 4: Run PCA analysis for aligned structural data
pc <- pca(pdbs)

# Generate score plots and loadings plots
plot(pc)
```



```
# Generate score plot of PC1 vs PC2
plot(pc, 1:2)
```

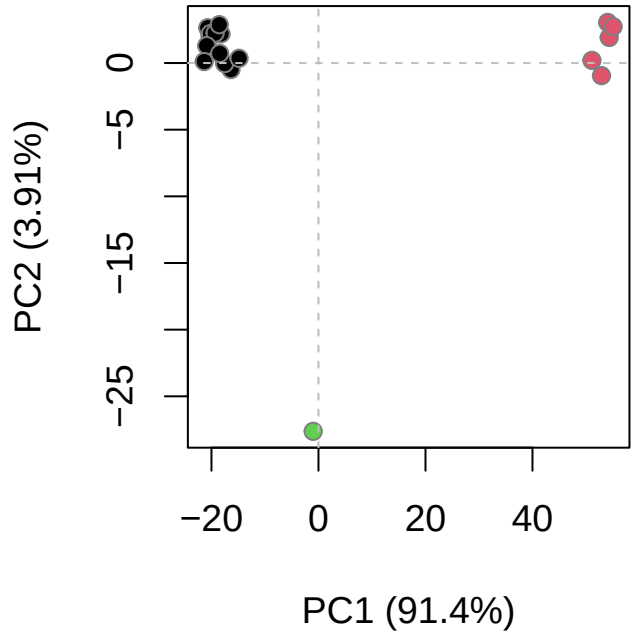


We can cluster structures using RMSD values to determine pairwise structural deviations:

```
# Calculate the RMSD values
rd <- rmsd(pdbs)
```

```
Warning in rmsd(pdbs): No indices provided, using the 182 non NA positions
```

```
# Group structures into 3 clusters using RMSD values
hc.rd <- hclust(dist(rd))
grps.rd <- cutree(hc.rd, k=3)

# Plot PC1 vs PC2, w/ points colored by RMSD-based clustering assignment
plot(pc, 1:2, col="grey50", bg=grps.rd, pch=21, cex=1)
```



The plot above depicts the separation and grouping of our collected PDB structures along PC1 and PC2, representing conformational differences among the structures.

PCA visualization

Interactive view of the PCA-captured structural differences:

```
view.pca(pc)
```

To view structural variation along PC1, in Mol*, first use `mktrj()` to generate a trajectory PDB file. The file can then be opened in Mol-star:

```
mktrj(pc, pc = 1, file = "pc_1.pdb")
```

Figure 7: Image of Mol-star animation of structural variation along PC1.